LegalEase

5. Project Development Phase

AI-Powered Legal Document Generator

Objective

Implement the LegalEase application components and connect frontend, backend, AI generation and document exports.

Backend Development

- Initialize the FastAPI application in main.py.
- Create a root health-check endpoint.
- Include routes from routes.py.
- Define the Pydantic DocumentRequest model.
- Create POST /generate.
- Call GeminiDocumentGenerator.generate_document(...).
- Return generated document text to the frontend.

AI Development

- Build a structured prompt from document type, parties, terms and effective date.
- Send the prompt to Gemini using generate_content.
- Receive and return generated legal document content.

Frontend Development

- Configure the Streamlit page.
- Provide inputs for document type, parties, terms and date.
- Send the request to the FastAPI backend.
- Display generated content.
- Allow editing in a text area.
- Provide TXT, DOCX and PDF download controls.

Formatting Development

- sanitize_text(text) removes special characters and typographic quotes for clean formatting.
- format_docx(text, doc_type) creates a formatted Word document with logo, headings, terms table and footer.

- format_pdf(text, doc_type) creates a branded PDF with logo, headings, bullet-style terms and footer.
- format_html_preview(text) converts output into styled HTML blocks.

Core User Inputs

- Document Type: examples include Agreement, Contract, NDA, Lease Agreement and Employment Offer Letter.
- Parties Involved: names and roles of the stakeholders.
- Terms & Conditions: clauses separated with semicolons.
- Effective Date: the date the agreement becomes legally valid.